Unidade 5

Backen con MongoDB. Instalación, configuración y uso práctico (II parte)

1. Backend con MongoDB. Instalación, configuración y uso práctico

Hasta ahora hemos desarrollado la parte cliente de nuestra aplicación utilizando **Vue.js**. Vue se encarga de la interfaz de usuario y de la interacción con el usuario, pero una aplicación completa necesita también un servidor que permita gestionar los datos de forma permanente.

Para ello vamos a incorporar un **backend** basado en **Node.js y Express**, que actuará como intermediario entre nuestra aplicación Vue.js y la base de datos.

La arquitectura que vamos a utilizar será:

Vue.js → Axios → Express → MongoDB

- **Vue.js** será responsable de la interfaz y de la lógica de la aplicación en el navegador.
- **Axios** permitirá realizar peticiones HTTP desde Vue hacia nuestro servidor.
- **Express** proporcionará una API REST para recibir y procesar esas peticiones.
- **MongoDB** será el sistema gestor de bases de datos donde almacenaremos la información de la aplicación.
- **Docker** nos permitirá ejecutar MongoDB de forma sencilla y aislada, sin necesidad de instalar el servidor de MongoDB directamente en el sistema operativo.

Además, utilizaremos un archivo ***.env*** para almacenar la configuración del backend, como la dirección de conexión con MongoDB o el puerto utilizado por Express. De esta forma evitamos escribir directamente estos datos en el código fuente y facilitamos la configuración de la aplicación en diferentes equipos.

En esta primera fase no vamos a desarrollar todavía el CRUD completo. El objetivo será preparar correctamente el entorno y conseguir que **Express pueda conectarse a MongoDB**, dejando preparada la estructura para incorporar posteriormente los modelos, las rutas y las operaciones de consulta, inserción, modificación y eliminación de datos.

MongoDB es un sistema gestor de **bases de datos NoSQL, orientado a documentos**. A diferencia de las bases de datos relacionales tradicionales, como MySQL o PostgreSQL, MongoDB no organiza la información mediante tablas y filas, **sino mediante documentos agrupados en colecciones.**

Los documentos de MongoDB utilizan un formato similar a JSON, aunque internamente se almacenan utilizando BSON (Binary JSON).

Por ejemplo, un paciente podría almacenarse como un documento:

```
{
  "dni": "12345678A",
  "nombre": "Juan",
  "apellidos": "García Pérez",
  "movil": "600123456",
  "email": "juan@correo.com"
}
```

En MongoDB podemos establecer una equivalencia aproximada con una base de datos relacional:

Bases de datos relacionales	MongoDB
Base de datos	Base de datos
Tabla	Colección
Fila o registro	Documento
Columna	Campo

Una de las principales características de MongoDB es que los documentos de una misma colección no tienen que tener exactamente los mismos campos. Esto proporciona una estructura flexible que resulta especialmente interesante para determinadas aplicaciones web.

SQL	MongoDB
`DATABASE`	Database
`TABLE`	Collection
Registro	Document
Campo	Field
`INSERT`	`insertOne()` / `insertMany()`
`SELECT`	`find()` / `findOne()`
`UPDATE`	`updateOne()` / `updateMany()`
`DELETE`	`deleteOne()` / `deleteMany()`

2. Configurador del gestor de BBDD MongoDB

Para trabajar con MongoDB vamos a utilizar Docker. De esta forma no es necesario instalar MongoDB directamente en el sistema operativo. MongoDB se ejecutará dentro de un contenedor.

Esta solución permite utilizar la misma configuración tanto en Ubuntu como en Windows, siempre que tengamos Docker instalado.

En **Ubuntu** instalaremos ***Docker Engine.*** en Ubuntu 24.04

1. **Abre una terminal y ejecuta los siguientes pasos.**

   sudo apt update

   sudo apt install ca-certificates curl

2. **Crear la carpeta para la clave de Docker:**

   sudo install -m 0755 -d /etc/apt/keyrings

3. **Descargar la clave oficial de Docker:**

   sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc

Y darle permisos:

sudo chmod a+r /etc/apt/keyrings/docker.asc

4. **Añadir el repositorio oficial de Docker:**

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF

Types: deb

URIs: https://download.docker.com/linux/ubuntu

Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")

Components: stable

Architectures: $(dpkg --print-architecture)

Signed-By: /etc/apt/keyrings/docker.asc

EOF

Después:

sudo apt update

Estos son los pasos que indica actualmente la documentación oficial de Docker para Ubuntu.

5. **Instalar Docker Engine y Docker Compose**

Este paso es importante para nuestro proyecto porque ya instalamos también Docker Compose:

sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

En **Windows** podemos utilizar **Docker Desktop**, que permite ejecutar contenedores Docker desde Windows. A veces pide Linux para Windows:

wsl –install

Una vez instalado Docker, podemos comprobar que funciona ejecutando:

docker --version

A continuación creamos el contenedor. Para ello crearemos un archivo:

docker-compose.yml

con el siguiente contenido:

```
services:
  mongodb:
    image: mongo:8.0
    container_name: mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: abc123
    ports:
      - "27017:27017"
    volumes:
```

```
    - mongodb_data:/data/db

volumes:

  mongodb_data:
```

Situados en la carpeta donde se encuentra *docker-compose.yml*, ejecutamos:

```
docker compose up -d
```

La opción -d hace que el contenedor se ejecute en segundo plano. Podemos comprobar que está funcionando con:

```
docker ps
```

Deberíamos encontrar un contenedor llamado:

```
mongodb
```

Con el contenedor funcionando, MongoDB estará disponible en:

```
localhost:27017
```

La cadena de conexión que utilizaremos posteriormente desde Node.js será:

```
mongodb://admin:abc123@localhost:27017/bbdd?authSource=admin
```

Donde:

- ***admin*** → usuario.
- ***abc123*** → contraseña.
- ***localhost*** → equipo donde se ejecuta Docker.
- ***27017*** → puerto de MongoDB.
- ***bbdd*** → base de datos que utilizaremos.
- ***authSource=admin*** → indica que el usuario administrador está creado en la base de datos admin.

MongoDB Compass. Para visualizar y gestionar las bases de datos podemos utilizar MongoDB Compass, una aplicación gráfica para MongoDB. Importante NO ES MONGODB solo una herramienta gráfica para ver las bases de datos. Se puede descargar aquí.

La conexión será:

mongodb://admin:abc123@localhost:27017/bbdd?authSource=admin

Seguimos con la configuración de server.js

Instalar ***dotenv*** que permite cargar variables de configuración desde un archivo .env de forma segura.

npm install dotenv

En el fichero .***env*** le añadimos inicialmente esto

API_URL="http://localhost:3000/api"

PORT=3000

MONGO_URI=mongodb://admin:abc123@localhost:27017/bbdd?authSource=admin

Modificar los parámetros en negrita:

```
import "dotenv/config";
import express from "express";
import fs from "fs";
import cors from "cors";


// Creamos la aplicación Express
const app = express();
```

```
app.use(cors());

//Usa el valor de PORT definido en las variables de entorno. Si no existe, usa 3000.

const PORT = process.env.PORT || 3000;


// Ruta de la API para obtener provincias y municipios

app.get("/api/municipios", (req, res) => {

  console.log("petición recibida");


  // Leemos el fichero JSON

  const datos = fs.readFileSync("./backend/data/municipios.json", "utf8");


  // Convertimos el texto JSON en un objeto JavaScript

  const datosJson = JSON.parse(datos);


  // Enviamos los datos como respuesta al cliente

  res.json(datosJson);

});


// Ponemos el servidor a escuchar

app.listen(PORT, () => {

  console.log(`Servidor funcionando en http://localhost:${PORT}`);
```

});

De momento no aparece para nada mongodb y es intencionado ya que primero comprobamos que .env funciona y que nuestro servidor sigue funcionando exactamente igual.

Es importante distinguir MongoDB del **driver de MongoDB.**

MongoDB es el servidor de bases de datos que estamos ejecutando dentro del contenedor. El paquete que instalaremos posteriormente con:

npm install mongodb

es el driver de **MongoDB para Node.js**. No instala otro servidor MongoDB; permite que nuestra aplicación Express pueda comunicarse con el MongoDB que ya tenemos ejecutándose en Docker.

Nos queda finalizar la configuración del server.js

```
import "dotenv/config";
import express from "express";
import fs from "fs";
import cors from "cors";
import { MongoClient } from "mongodb";  //importación


// Creamos la aplicación Express
const app = express();
app.use(cors());


// Usa el valor de PORT definido en las variables de entorno.  Si no existe, usa 3000.
```

```
const PORT = process.env.PORT || 3000;


// URL de conexión a MongoDB
const MONGO_URI = process.env.MONGO_URI;

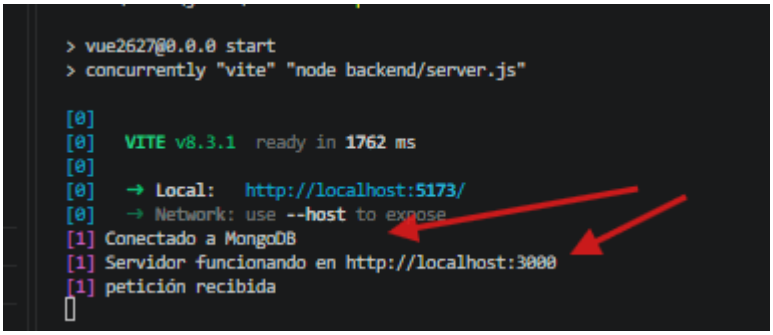

// Creamos el cliente de MongoDB
const client = new MongoClient(MONGO_URI);


// Ruta de la API para obtener provincias y municipios
app.get("/api/municipios", (req, res) => {
  console.log("petición recibida");
  // Leemos el fichero JSON
  const datos = fs.readFileSync(
    "./backend/data/municipios.json",  "utf8"
  );

  // Convertimos el texto JSON en un objeto JavaScript
  const datosJson = JSON.parse(datos);

  // Enviamos los datos como respuesta al cliente
  res.json(datosJson);
});
```

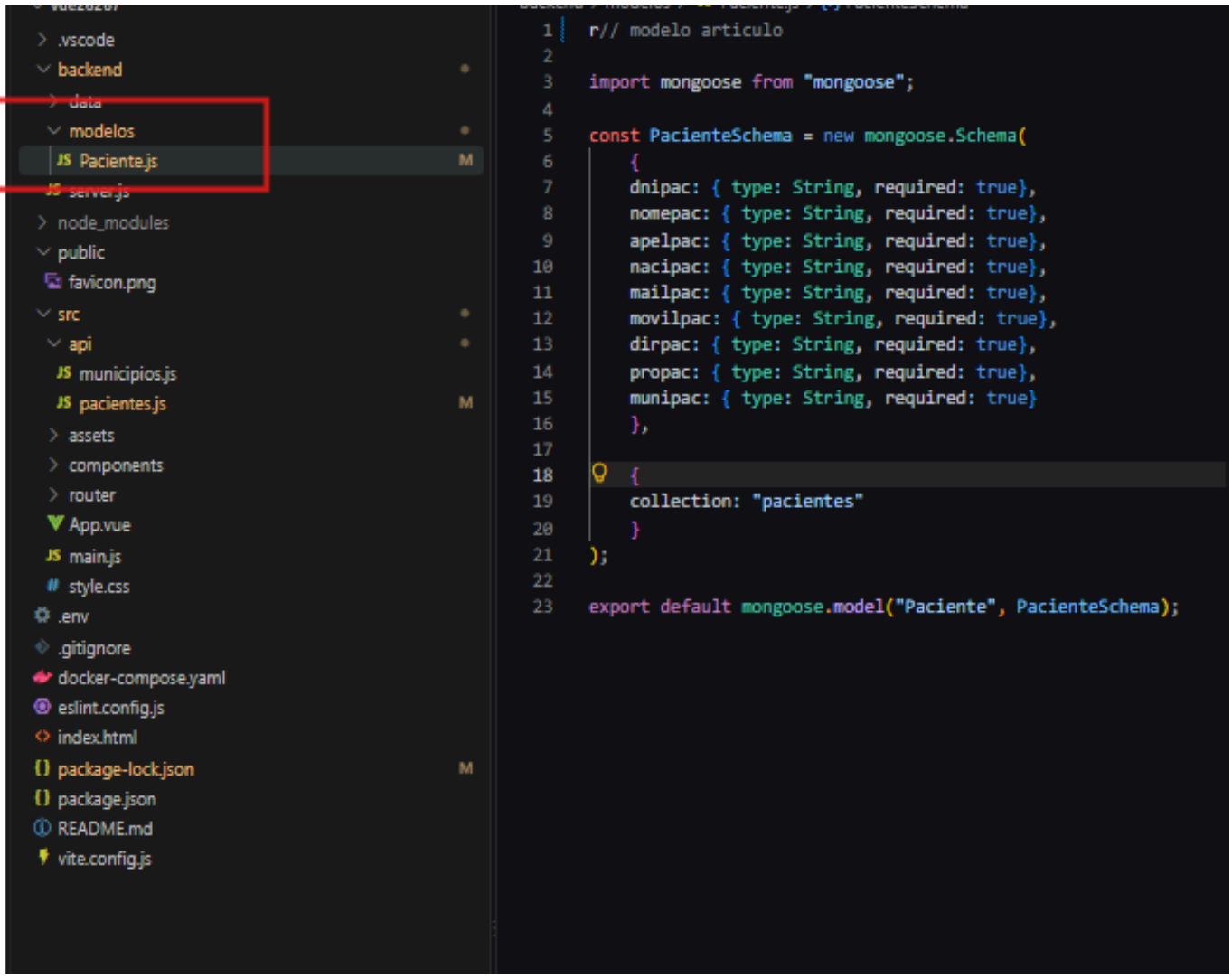
```
// Conectamos con MongoDB y arrancamos el servidor
async function iniciarServidor() {
  try {
    // Conectamos con MongoDB
    await client.connect();
    console.log("Conectado a MongoDB");
    // Ponemos el servidor a escuchar
    app.listen(PORT, () => {
      console.log(
        `Servidor funcionando en http://localhost:${PORT}`
      );
    });
  } catch (error) {
    console.error("Error al conectar con MongoDB:", error);
  }
}
// Iniciamos el servidor llamando a la función.
iniciarServidor();
```



3. Uso práctico del gestor MongoDB

Vamos a llevar a cabo la configuración del formulario para que pueda guardar, obtener, modificar y eliminar, en este caso pacientes.

1. Creamos una carpeta llamada **modelos dentro de backend** para crear una “plantilla” de lo que queremos guardar en este caso datos del Paciente.



2. A continuación creamos las rutas para el backend. **No confundir con el vue-router del frontend.** Son cosas diferentes: el primero indica qué función debe ejecutar, el segundo nos permite movernos por el menú de la aplicación. Para ello creamos una carpeta en el backend llamada ***rutas*** y dentro las rutas de las funciones que permiten guardar, modificar, borrar o insertar un paciente así como sus respectivos códigos.

```
import express from "express";

import Paciente from "../modelos/Paciente.js";

const router = express.Router();

// Obtener todos

router.get("/", async (req, res) => {

    try {

        const pacientes = await Paciente.find();

        res.json(pacientes);

    } catch (error) {

        res.status(500).json({

            mensaje: "Error al obtener pacientes"

        });

    }

});

// Obtener uno

router.get("/:dni", async (req, res) => {

    try {

        const paciente = await Paciente.findOne({

            dnipac: req.params.dni

        });
```

```
        if (!paciente) {
            return res.status(404).json({
                mensaje: "Paciente no encontrado"
            });
        }
        res.json(paciente);
    } catch (error) {
        res.status(500).json({
            mensaje: "Error al obtener el paciente"
        });
    }
});
// Crear
router.post("/", async (req, res) => {
    try {
        const paciente = new Paciente(req.body);
        const nuevoPaciente = await paciente.save();
        res.status(201).json(nuevoPaciente);

    } catch (error) {
        res.status(500).json({
            mensaje: "Error al crear el paciente"
        });
    }
```

```
});

// Modificar

router.put("/:dni", async (req, res) => {

    try {

        const paciente = await Paciente.findOneAndUpdate(

            { dnipac: req.params.dni },

            req.body,

            { new: true }

        );

        if (!paciente) {

            return res.status(404).json({

                mensaje: "Paciente no encontrado"

            });

        }

        res.json(paciente);



    } catch (error) {



        res.status(500).json({

            mensaje: "Error al modificar el paciente"

        });

    }

});

// Eliminar
```

```
router.delete("/:dni", async (req, res) => {
    try {
        const paciente = await Paciente.findOneAndDelete({
            dnipac: req.params.dni
        });
        if (!paciente) {
            return res.status(404).json({
                mensaje: "Paciente no encontrado"
            });
        }
        res.json({
            mensaje: "Paciente eliminado"
        });
    } catch (error) {
        res.status(500).json({
            mensaje: "Error al eliminar el paciente"
        });
    }
});
export default router;
```

Una puntualización. En códigos y **aplicaciones más compleja**s algunos desarrolladores declaran las funciones en ***pacientes.rutas.js*** y luego codifican las mismas en *pacientes.controller.js* situada en una carpeta llamada controller (el nombre puede cambiar, controller es un ejemplo). Como no es un proyecto muy complejo, nosotros por comodidad fusionamos ambos ficheros en uno solo.

3. Tenemos que decirle a **server.js** donde tiene que buscar estos códigos y las rutas necesarias:

```
import express from "express";
import fs from "fs";
import cors from "cors"; //evita bloqueos entre servidores
import "dotenv/config";
import mongoose from "mongoose";

import pacientesRutas from './rutas/pacientes.rutas.js';
//importa el modelo de paciente
// Creamos la aplicación Express
const app = express();
app.use(cors());
app.use(express.json()); //para que pueda leer json
app.use('/api/pacientes', pacientesRutas); //usa el modelo de paciente
//USA EL PUERTO definido en la variables de entorno y si no coge el 3000
const PORT = process.env.PORT || 3000;
//URL conexion con mongodb
const MONGO_URI = process.env.MONGO_URI;

// Ruta de la API para obtener provincias y municipios
app.get("/api/municipios", (req, res) => {

  // Leemos el fichero JSON
  const datos = fs.readFileSync("./backend/data/municipios.json", "utf8");

  // Convertimos el texto JSON en un objeto JavaScript
```

```
  const datosJson = JSON.parse(datos);


  // Enviamos los datos como respuesta al cliente
  res.json(datosJson);
});


// Ponemos el servidor a escuchar en el puerto 3000


async function iniciaServer(){
   try{
     //conectamos con mongodb
     await  mongoose.connect(MONGO_URI);
     console.log("Conectado a MongoDB");
     app.listen(PORT, () => {
     console.log(`Servidor funcionando en http://localhost:${PORT}`);
      });
   } catch(error){
     console.error("error de conexion", error)
   }
 }
iniciaServer()
```

4. Nos vamos al frontend y empezamos por el **api/pacientes.js** que será el encargado de hacer peticiones a ***server.js*** De momento, insertar y obtener pacientes:

```
import axios from "axios"

//en produccion se usa la url del servidor

//const API_URL = "https://servidor.com/api";



const API_URL = "http://localhost:3000/api";



// Guardar paciente

export async function savePaciente(paciente) {

    const res = await axios.post(`${API_URL}/pacientes`, paciente);

    return res.data;

}
```

```
// Obtener todos los pacientes

export async function getPacientes() {

    const res = await axios.get(`${API_URL}/pacientes`);

    return res.data;

}
```

5. Nos queda el **componente *XestionPacientes***. Aquí básicamente es tocar solo la ***zona de script*** y un par de detalles del ***template.***

A. Primero importar los dos módulos con los que estoy trabajando.

import { getPacientes, savePaciente } from "../api/pacientes.js";

B. Cambiamos usuarios del programa inicial por pacientes

const pacientes = ref([]);

Y eliminamos el usuario que hay en ***unMounted()***. En su lugar hacemos una petición a la base de datos.

pacientes.value = await obtener Pacientes();

C. El cambio más importante está en ***guardarPaciente***. Ahora ya no añadimos a un array añadimos a la base de datos usando las funciones.

```
async function guardarPaciente() {
    try {
        //IMPORTANTE SINO CARGA LOS ID Y NO LOS NOMBRES
        // tomar el nombre del municipio seleccionado
        // y asignarlo a novoPaciente.munipac
        // y de la provincia seleccionado
        // y asignarlo a novoPaciente.propac
        const provincia = provincias.value.find(
          p => p.id === novoPaciente.propac
        );
        const municipio = municipios.value.find(
        m => m.id === novoPaciente.munipac
      );

      novoPaciente.propac = provincia.nm;
      novoPaciente.munipac = municipio.nm;
```

```
        const pacienteGuardado = await savePaciente(novoPaciente);
        pacientes.value.push(pacienteGuardado);
        console.log("Paciente gardado correctamente");
    } catch (error) {
        console.error("Error ao gardar paciente:", error);
        }
    }
```

D. Nos vamos al template y en el ***form*** cambiamos por ***guardarPaciente.***

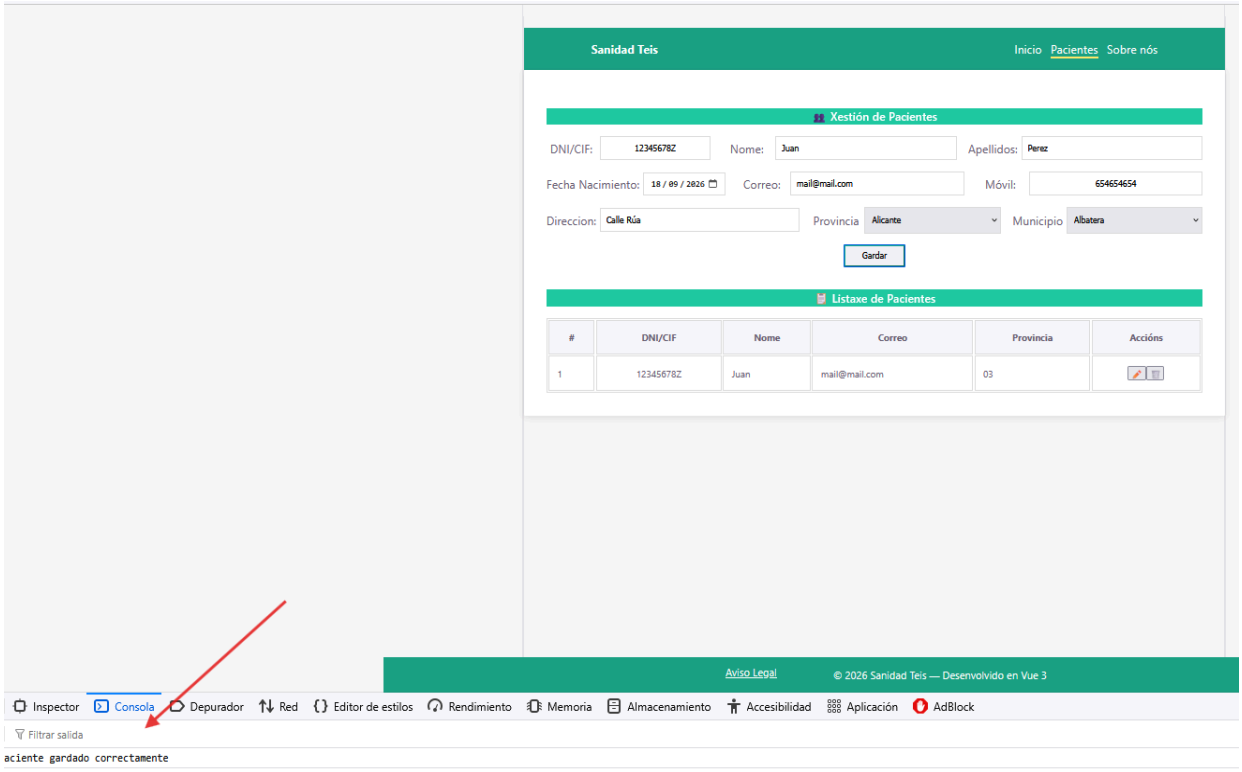
E. En la tabla cambiamos usuarios por **pacientes**.

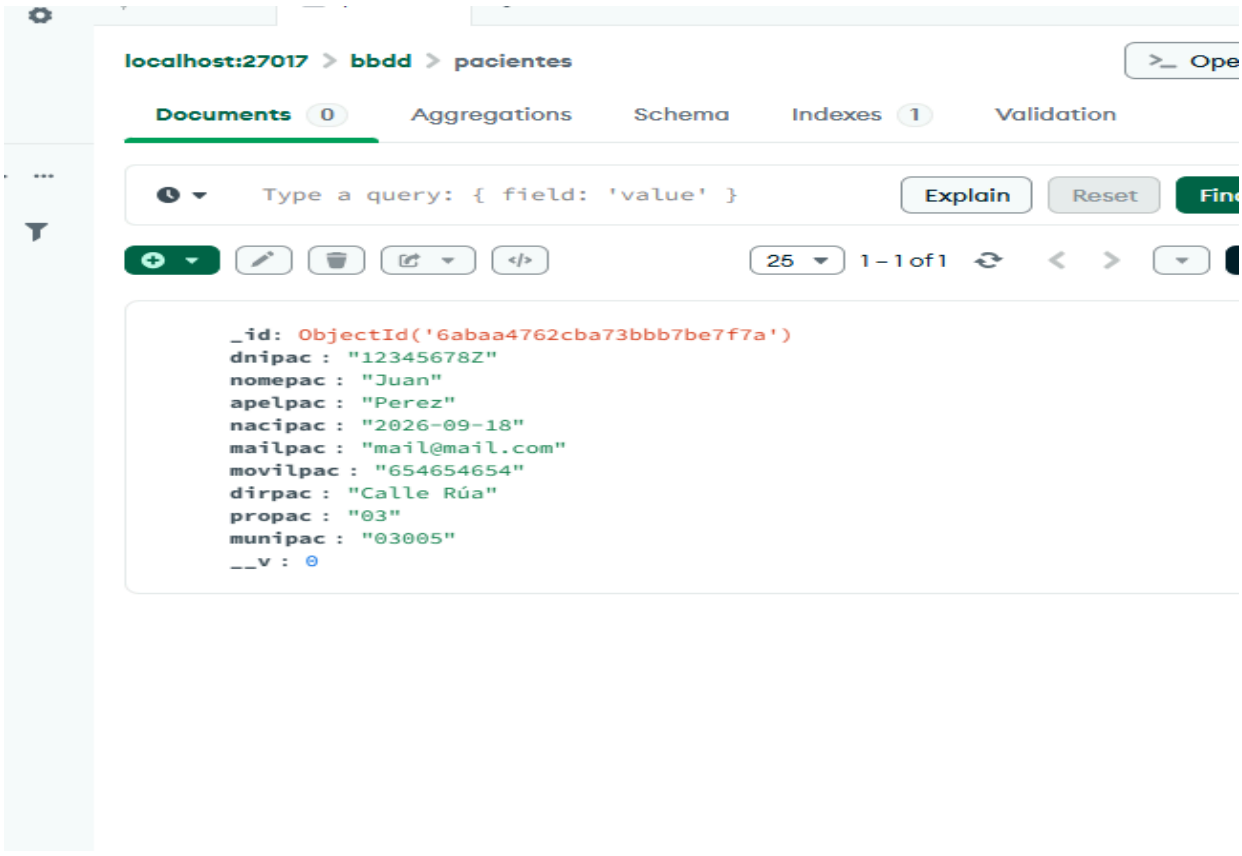
F. Para evitar confusiones tenemos que cambiar los nombres de los campos de novoPaciente para que coincida con el modelo Paciente.

```
const novoPaciente = reactive({
    dnipac: "",
    nomepac: "",
    apelpac: "",
    nacipac: "",
    mailpac: "",
    movilpac: "",
    dirpac: "",
    propac: "",
    munipac: ""
});
```

Y **revisamos estos campos en el template** para que coincidan.

Pruebas:


Sanidad Teis
Inicio Pacientes Sobre nós
Xestión de Pacientes
DNI/CIF: 12345678Z
Nome: Juan
Apellidos: Perez
Fecha Nacimiento: 18 / 09 / 2026
Correo: mail@mail.com
Móvil: 654654654
Direccion: Calle Rúa
Provincia Alicante
Municipio Albatera
Gardar
Listaxe de Pacientes
DNI/CIF Nome Correo Provincia Accións
1 12345678Z Juan mail@mail.com 03
Aviso Legal
© 2026 Sanidad Teis — Desenvolvido en Vue 3
Inspector Consola Depurador Red Editor de estilos Rendimiento Memoria Almacenamiento Accesibilidad Aplicación AdBlock
Filtrar salida
aciente gardado correctamente


localhost:27017 > bbdd > pacientes
Documents 0
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' }
Explain
Reset
25
1 – 1 of 1
_id: ObjectId('6abaa4762cba73bbb7be7f7a')
dnipac : "12345678Z"
nomepac : "Juan"
apelpac : "Perez"
nacipac : "2026-09-18"
mailpac : "mail@mail.com"
movilpac : "654654654"
dirpac : "Calle Rúa"
propac : "03"
munipac : "03005"
__v : 0